Datazoic Agentic Platform

Deep Technical & Engineering Specification

1. Technology Stack & Component Architecture

The Datazoic Agentic Platform is engineered in Python 3.10 with high-performance asynchronous micro-frameworks:

- **Backend Framework:** FastAPI 0.110+ with Uvicorn ASGI server and Pydantic v2 data validation.
- **Orchestration Engine:** LangGraph 0.2+ compiled StateGraph with explicit typed transitions.
- **Vector Database:** Qdrant Client (In-memory mode with 768-dimensional cosine distance collections).
- **Relational Metadata DB:** SQLAlchemy 2.0 AsyncIO with SQLite (local development) and PostgreSQL (production).
- **Frontend Interface:** Streamlit 1.32+ communicating exclusively through authenticated REST endpoints.
- **LLM Integration:** Google Gemini 2.5 Flash with deterministic fallback embedding subsystems.

2. Relational Database Schema Specification

The relational database schema implements six core tables defined in app/db/models.py:

Table Name	Primary Key	Key Attributes	Relationships
users	user_id (UUID)	email, hashed_password, role (admin/user), is_active	One-to-Many: agent_runs, token_usage
tool_definitions	tool_id (UUID)	name, namespace, method, path, risk_level, approval_policy	Unique: name
knowledge_sources	source_id (UUID)	name, source_type, domain, chunk_count, status	Indexed in Qdrant KB
execution_logs	log_id (UUID)	run_id, user_id, tool_name, latency_ms, status	Foreign Key: user_id
agent_runs	run_id (UUID)	user_id, query, status, approval_status, execution_trace	Foreign Key: user_id
token_usage	usage_id (UUID)	run_id, user_id, input_tokens, output_tokens, total_tokens	Foreign Key: user_id

3. Canonical AgentState Schema Definition

The complete conversational context is encapsulated within the canonical AgentState TypedDict (app/graph/state.py):

```
class AgentState(TypedDict, total=False): # Identifiers & Context request_id: str; task_run_id: str; thread_id: str; user_id: str; user_role: str user_prompt: str; intent: Optional[str]; sub_intents: List[str] selected_registries: List[str] # Candidate Retrieval & Planning candidate_tool_ids: List[str]; candidate_tools: List[Dict[str, Any]] plan: List[Dict[str, Any]]; current_step_index: int pending_tool_call: Optional[Dict[str, Any]] # HITL Policy & Approval approval_status: str; approval_request_id: Optional[str] pending_approval: Optional[Dict[str, Any]] # Execution & Multi-Step DAG Loop execution_log: List[Dict[str, Any]]; execution_trace: List[Dict[str, Any]] last_result: Optional[Dict[str, Any]]; has_next_step: bool iteration_count: int; max_iterations: int # Observability & Tokens timeline_events: List[Dict[str, Any]]; token_usage: Dict[str, int]
```

4. Automated Evaluation & Verification Suite

System correctness is formally verified by the automated evaluation test runner in tests/evals/run_evals.py. The suite executes eight golden queries from golden_queries.jsonl verifying intent extraction, candidate retrieval, HITL policy enforcement, and multi-step execution. **Current Pass Rate: 100.0% (8/8 Passed)**:

ID	Query Text	Expected Intent	Policy Check	Result
eval_01	calculate 450 * 0.18 + 25.50	CALCULATION	Auto-approved	PASS (1641ms)
eval_02	what is the current time in UTC?	TIME	Auto-approved	PASS (40ms)
eval_03	draft an email to client@example.com about overdue invoice	COMMUNICATION	Auto-approved	PASS (26ms)
eval_04	send email to billing@example.com with body Please confirm receipt	COMMUNICATION	HITL Pending	PASS (49ms)
eval_05	create an invoice for client@example.com	INVOICING	Clarification Gate	PASS (26ms)
eval_06	create an invoice for $350.00 to user@example.com	INVOICING	HITL Pending	PASS (36ms)
eval_07	what are the dispute policies and fees for chargebacks?	KNOWLEDGE_RAG	Auto-approved	PASS (39ms)
eval_08	create an invoice for $500.00 to finance@example.com and send it	INVOICING	HITL Pending	PASS (137ms)